

Using Wireshark to Investigate the Transmission of Dummy Login Credentials over HTTP and HTTPS

Cryptography is an important part of network security. It protects sensitive information such as usernames, passwords, banking details, and personal data from unauthorized access.

HTTP (Hypertext Transfer Protocol) does not provide encryption. Therefore, information transmitted through HTTP can be observed in readable form when network traffic is captured.

HTTPS (Hypertext Transfer Protocol Secure) uses TLS (Transport Layer Security) to encrypt the communication between the client and server. As a result, sensitive information such as login credentials cannot be directly read from the captured network packets.

In this experiment, a controlled environment was created in Kali Linux using a dummy login page. Wireshark was used to capture and analyze the traffic generated during login over HTTP and HTTPS.

Tools

- Kali Linux
- Oracle VirtualBox
- Wireshark
- Python 3
- OpenSSL
- Firefox Web Browser
- Dummy username and password

Dummy Credentials Used

Username: testuser

Password: Test@123

Only dummy credentials were used in this controlled experiment.

HTTP

HTTP stands for Hypertext Transfer Protocol. It is used for communication between a web browser and a web server.

HTTP does not encrypt the transmitted data. Therefore, if sensitive information such as login credentials is sent through HTTP, the information may be visible in captured network traffic.

The standard HTTP port is **80**. In this experiment, port **8080** was used for the local dummy HTTP server.

HTTPS

HTTPS stands for Hypertext Transfer Protocol Secure. It is HTTP communication protected using TLS (Transport Layer Security).

TLS encrypts the data exchanged between the client and server. Therefore, even if network packets are captured using Wireshark, the actual application data cannot normally be read without the appropriate cryptographic keys.

The standard HTTPS port is **443**. In this experiment, port **8443** was used for the local HTTPS server.

Role of Encryption

Encryption converts readable information, called plaintext, into an unreadable form called ciphertext.

For example:

Plaintext:

username=testuser & password=Test@123

After encryption, the transmitted data appears as unintelligible encrypted information.

This prevents unauthorized users who capture network traffic from directly reading sensitive information.

Experimental Setup

The experiment was performed entirely in a controlled Kali Linux environment.

A dummy login server was created locally using Python. The server was accessed using the loopback address:

127.0.0.1

Two separate experiments were performed:

1. HTTP login using port **8080**
2. HTTPS login using port **8443**

Wireshark was used to capture the traffic on the **Loopback (lo)** interface.

Experiment 1 – HTTP Traffic Analysis

Procedure

1. A local dummy HTTP login server was created using Python.
2. The server was started at:

`http://127.0.0.1:8080`

3. Wireshark was opened.
4. The **Loopback: lo** interface was selected for packet capture.
5. The following Wireshark display filter was used:

`tcp.port == 8080`

6. The dummy login page was opened in Firefox.
7. The following dummy credentials were entered:

Username: testuser

Password: Test@123

8. The login form was submitted.
9. The captured packets were analyzed in Wireshark.

Observation

The HTTP POST request was visible in Wireshark.

The packet details clearly displayed the form data:

- username = testuser
- password = Test@123

This shows that the login credentials were transmitted in readable form because HTTP does not encrypt the data.

Screenshot – HTTP Traffic

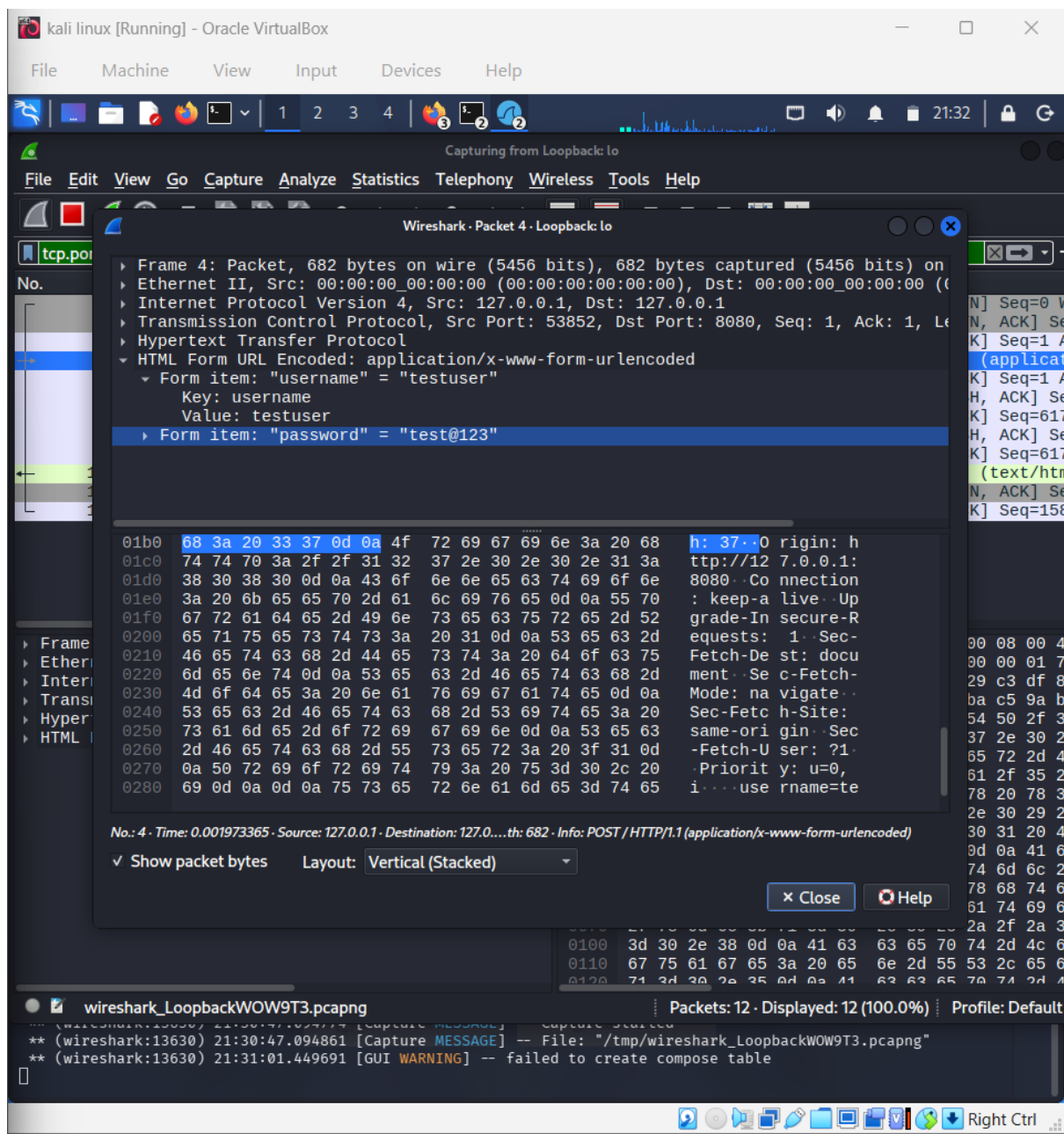


Figure 1: Wireshark capture showing dummy login credentials transmitted in readable form over HTTP.

Experiment 2 – HTTPS Traffic Analysis

Procedure

1. A self-signed certificate was generated using OpenSSL in the controlled Kali Linux environment.
2. A local HTTPS dummy login server was created using Python.
3. The server was started at:

`https://127.0.0.1:8443`

4. Wireshark was opened and the **Loopback: lo** interface was selected.
5. The following Wireshark display filter was used:

`tcp.port == 8443`

6. The HTTPS login page was opened in Firefox.
7. The same dummy credentials were entered:

Username: testuser

Password: Test@123

8. The login form was submitted.
9. The captured packets were analyzed in Wireshark.

Observation

The captured traffic showed **TLSv1.3** packets and **Application Data**.

Unlike the HTTP experiment, the username and password were not displayed as readable form data.

The captured application data appeared as encrypted/binary data.

This demonstrates that HTTPS protects sensitive information by encrypting the communication using TLS.

Screenshot – HTTPS Traffic

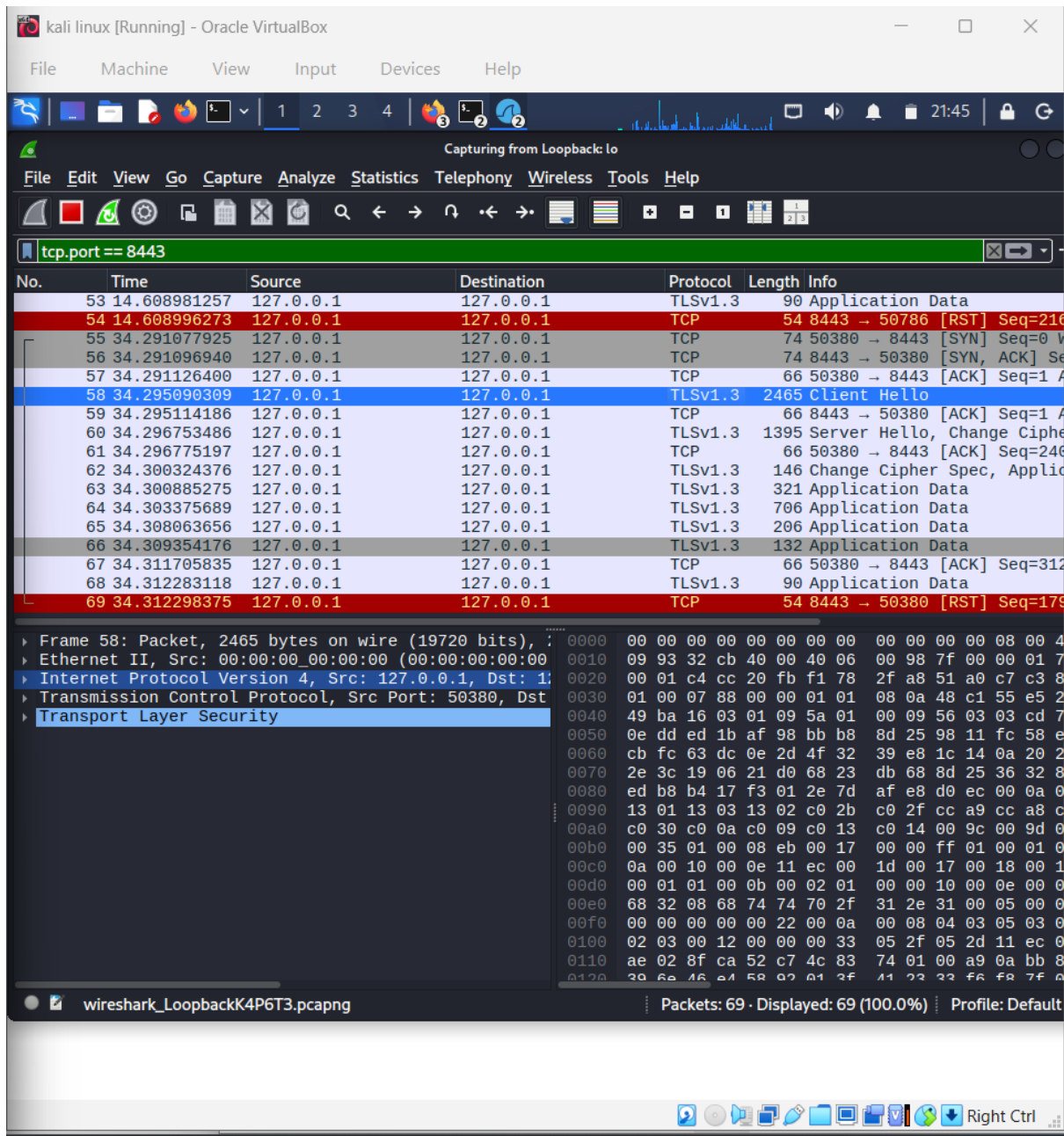


Figure 2: Wireshark capture showing encrypted TLS traffic during HTTPS login.

Comparison Between HTTP and HTTPS

Feature	HTTP	HTTPS
Full form	Hypertext Transfer Protocol	Hypertext Transfer Protocol Secure
Encryption	No	Yes
Security	Less secure	More secure
Standard port	80	443
Port used in experiment	8080	8443
Wireshark traffic	HTTP	TLSv1.3
Credentials visible	Yes	No
Sensitive data protection	Not provided	Provided through TLS encryption

Result

The experiment successfully demonstrated that:

- HTTP transmits login information without encryption.
- Dummy credentials sent over HTTP were visible in readable form in Wireshark.
- HTTPS uses TLS encryption to protect transmitted information.
- Dummy credentials sent over HTTPS were not directly visible in the captured traffic.
- Encryption helps protect sensitive information from being exposed through network traffic capture.